LesionNet: Lesion-Gated Classification for Background-Robust, Faithfully Explainable and Severity-Aware Plant Disease Diagnosis

Raj Naik, Dhruv Patel, Shreya Mehta, Parshva Mathukia and Prachi Tawde
Department of Information Technology
Dwarkadas J. Sanghvi College of Engineering, Mumbai, India
Corresponding author: Raj Naik (srnaiksir926@gmail.com)

***Abstract*—Deep networks trained on PlantVillage routinely report near-perfect accuracy, but it is hard to tell whether they have learned the disease or the conditions under which each class was photographed. We tested four fine-tuned CNNs on a held-out split of 3,109 images covering 15 tomato, potato and pepper classes. Only 7–12% of their Grad-CAM++ evidence fell on diseased tissue, while 25–49% fell on the background, and ResNet50, the most accurate of the four at 99.8%, dropped to 45.1% once the same leaves were placed on plain grey. We propose LesionNet, a segment-then-classify network. A segmentation head labels each pixel as background, healthy leaf or lesion, and the classifier is given nothing except features averaged over the predicted lesion and leaf pixels, plus the predicted lesion area. We call this lesion-gated pooling. Because the gate is detached from the classification loss, the lesion map the user sees is the input to the decision rather than an explanation produced afterwards, and the same map gives a severity estimate. No image was annotated by hand: training uses colour-based pseudo-masks and a background-swap augmentation. With an EfficientNetB0 backbone, LesionNet reaches 99.5% on clean images and 98.7% on grey backgrounds, where an identically trained plain EfficientNetB0 manages 67.5%. Its strongest evidence lies on a lesion in 97% of images (CNNs: 49–64%), and its predicted diseased area tracks the measured area with Spearman $\rho = 0.90$. On 1,012 PlantDoc field photographs, however, every model we trained, LesionNet included, scored between 19.6% and 23.6%, so removing the background shortcut alone does not make a model ready for the field. LesionNet is the default model in AGRI-X, our open-source decision-support app for farmers.**

***Index Terms*—plant disease detection, shortcut learning, explainable AI, interpretable-by-design, weak supervision, lesion segmentation, severity estimation, PlantVillage, PlantDoc**

I. Introduction

For many farmers the first attempt at diagnosing a crop disease is a photo taken on a phone. Since PlantVillage was released [1], transfer-learned CNNs have reported 97–99% accuracy on its leaf images [2]–[4], yet tools built on them have been far less reliable in real fields [5], [6]. One reason is the way the dataset was collected. Classes differ in background, lighting and capture session, and a model can pick up on those cues instead of the symptoms [7]. Geirhos *et al.* call this shortcut learning [8], and background-swap tests have exposed the same problem in general object recognition [9].

We think a diagnostic model for farmers has to meet three requirements. The prediction should depend on the diseased tissue and not on the table under the leaf, which Ross *et al.* describe as being right for the right reasons [10]. The explanation shown to the user should also be faithful to what the model actually did. Grad-CAM [11] and Grad-CAM++ [12] only approximate where a network looked; they can be insensitive to the model they are supposed to explain [13], and for the backbones we use they are computed on a 7×7 grid. Rudin argues that high-stakes decisions call for models that are interpretable by design [14]. Finally, the farmer needs to know how badly the leaf is affected, since that decides the treatment, and a class label alone does not say [15].

Our approach rests on one constraint: the classifier is only allowed to see what the network itself has outlined as leaf or lesion. This paper makes the following contributions.

1) We measure how four standard CNNs (MobileNetV2, ResNet50, EfficientNetB0 and DenseNet121) spread their evidence across a leaf and how they cope when the background is changed, replaced or the leaf is removed. The most accurate of them turns out to be the least robust. Along the way we found a Grad-CAM pitfall in Keras 3 that silently corrupts explanations of nested models.
2) We introduce LesionNet. Class features are pooled only over predicted lesion and leaf pixels, and a stop-gradient on the gate prevents the classification loss from pushing the gate towards useful background regions. The lesion map is therefore, by construction, the evidence used for the decision.
3) LesionNet is trained from colour-based pseudo-masks, with healthy-class images as negatives, reliability weights and background-swap augmentation. No pixel was labelled by hand.
4) LesionNet matches the best CNN on clean images. Against an identically trained plain backbone it makes 25 times fewer errors on grey backgrounds, and its lesion area predicts severity with $\rho = 0.90$, while Grad-CAM areas show no relationship ($|\rho| \leq 0.13$).
5) On PlantDoc field photographs [16], every PlantVillage-trained model, ours included, falls to 19.6–23.6%. We report this negative result in full and release the code, results and the AGRI-X application.

II. Related Work

Plant disease classification. Mohanty *et al.* [2] reached 99.35% on PlantVillage with transfer learning, and Ferentinos [3] and Too *et al.* [4] reported similar figures with deeper architectures. Barbedo [5], [6] pointed to the limited variety of training images and the controlled capture conditions as the main reasons these models struggle in the field. PlantDoc [16] collected field images from the internet and found large accuracy drops for models trained on lab photos.

Dataset bias and shortcut learning. Noyan [7] showed that PlantVillage classes can be predicted well above chance from background pixels alone. Geirhos *et al.* [8] review shortcut learning more broadly, and Xiao *et al.* [9] used background swaps to probe ImageNet models. We apply the same kind of test to leaf images using automatic leaf masks, and we add a variant in which the leaf silhouette is also removed.

Explanations and interpretable models. Most plant pathology papers explain their models with Grad-CAM [11] or Grad-CAM++ [12], even though saliency maps can fail simple sanity checks [13]. Localisation is often scored with the pointing game [17]. Prototype networks [18] and attention branch networks [19] are interpretable by design; in the latter, attention re-weights the features, but background features can still reach the classifier. Ross *et al.* [10] penalise input gradients outside annotated regions, which needs masks during training. LesionNet takes background features out of the classifier input altogether and produces its own masks automatically.

Lesion segmentation and severity. Wang *et al.* [15] predicted severity stages directly from images. Pixel-level lesion datasets such as PlantSeg [20] have appeared only recently and are expensive to build. Our masks follow the pseudo-labelling idea [21], except that they come from a colour model refined with GrabCut [22] rather than from a teacher network.

III. Method

A. Problem setting

The model receives an RGB leaf image x and predicts one of $K{=}15$ classes (Table I). Every training image has a class label y and an automatically generated label map $m \in \{0,1,2\}^{H\times W}$ for background, healthy leaf and lesion, together with a reliability weight $w \in \{0,1\}$. We used no manual pixel labels at any point.

B. Annotation-free pseudo-masks

The masks come from a deterministic colour pipeline that we originally wrote for the AGRI-X app. We froze it before running any LesionNet experiment. It works in four steps.

1) *Leaf.* A background colour model is estimated from a band along the image border, excluding clearly green pixels. Plant-coloured pixels (green, yellow-green, brown with sufficient saturation) seed the leaf; pale leaves are recovered by an Otsu threshold on greenness ($-a^*$ in CIELAB). The coarse mask is refined with GrabCut [22]. Dark necrotic tissue that touches the leaf and differs in chroma from the background is added back, the largest component is kept and holes are filled.
2) *Healthy reference.* The healthy colour of *this* leaf is the median (V, S, a^*, b^*) of clearly green, well-lit pixels (HSV hue 34–88, $S, V \geq 50$), inside the leaf minus a 2.5% rim.
3) *Lesions.* Pixels are necrotic if they are brown or red-brown and darker than the reference, very dark, or shifted by more than 14 units towards red in a^*. They are chlorotic if they are yellow or measurably yellower than the reference hue. For the two classes with white fungal growth (late blight, leaf mould) a pale class is added.
4) *Clean-up.* Morphological opening and closing remove specks. Components smaller than 0.08% of the leaf are discarded, as are strips along the leaf outline whose colour matches the adjacent surface (cast shadows, background bleed).

In the 3,221 healthy-class images, any pixel marked as lesion is relabelled as healthy leaf, which gives the network clean negative examples. A diseased image gets $w{=}1$ only when the pipeline's own reliability checks pass (the leaf covers 12–97% of the frame and enough healthy tissue is left as a reference) and at least 1% of the leaf is lesion. Otherwise $w{=}0$ and the image only trains the classifier, so the network is never taught that a sick leaf is healthy. This leaves 17,895 of the 20,638 images (86.7%) with segmentation supervision. Lesions cover a median of 8.8% of the leaf (mean 12.8%), and the leaf takes up 42.5% of the image on average.

C. Architecture

LesionNet (Fig. 1) is built on an ImageNet-pretrained [23] backbone, EfficientNetB0 [24] by default or MobileNetV2 [25], from which we take features at strides 8, 16 and 32. A small FPN-style decoder [26] turns these into class probabilities $P \in [0,1]^{56\times56\times3}$. It uses $1{\times}1$ lateral convolutions to 128 channels, bilinear top-down addition, two $3{\times}3$ conv-BN-ReLU blocks and a final $2\times$ upsampling. For classification, stride-16 features are concatenated with upsampled stride-32 features and projected to $F \in \mathbb{R}^{14\times14\times256}$.

D. Lesion-gated pooling

Let $\tilde{P}$ be P area-averaged to $14{\times}14$ ($4{\times}4$ average pooling) and passed through a stop-gradient $\mathrm{sg}(\cdot)$. With lesion gate $g_\ell = \tilde{P}_{\text{lesion}}$ and leaf gate $g_f = \tilde{P}_{\text{leaf}} + \tilde{P}_{\text{lesion}}$,

$$\bar{f}_\ell = \frac{\sum_u g_\ell(u)\, F(u)}{\sum_u g_\ell(u) + \epsilon}, \qquad \bar{f}_f = \frac{\sum_u g_f(u)\, F(u)}{\sum_u g_f(u) + \epsilon}, \quad (1)$$

$$a = \frac{\sum_u g_\ell(u)}{\sum_u g_f(u) + \epsilon}, \qquad z = [\, \bar{f}_\ell \,\|\, \bar{f}_f \,\|\, a \,] \in \mathbb{R}^{513}, \quad (2)$$

with $\epsilon = 10^{-3}$. The vector z goes through dropout (0.3), a 256-unit ReLU layer and a softmax.

Positions that the segmentation head assigns to background ($g_f(u) \approx 0$) add nothing to z, so the class is computed from leaf and lesion positions only. Features at those positions still have receptive fields reaching past the leaf edge, a point we return to in Sec. VI. The map shown to the user, P_{lesion}, is the same weighting the classifier uses [14]. The ratio a is the

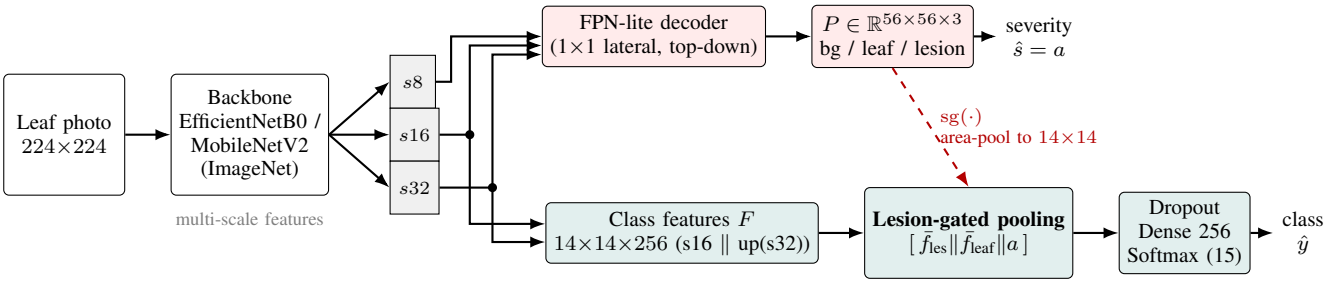


Fig. 1. LesionNet. A segmentation head labels each pixel as background, healthy leaf or lesion. The classifier sees only features averaged over predicted lesion pixels ($\bar{f}_{\text{les}}$) and leaf pixels ($\bar{f}_{\text{leaf}}$), plus the lesion-to-leaf area ratio a. The gate is detached (sg = stop-gradient), so only the pseudo-masks shape it. The same map is shown to the user and gives the severity $\hat{s}$.

TABLE I
DATASET: THE 15 PLANTVILLAGE CLASSES USED (20,638 IMAGES), SPLIT 70/15/15 PER CLASS (SEED 42) INTO 14,440 / 3,089 / 3,109 IMAGES.

Crop	Class	n	Crop	Class	n
Pepper	Bacterial spot	997	Tomato	Bacterial spot	2127
Pepper	Healthy	1478	Tomato	Early blight	1000
Potato	Early blight	1000	Tomato	Late blight	1909
Potato	Late blight	1000	Tomato	Leaf mould	952
Potato	Healthy	152	Tomato	Septoria spot	1771
Tomato	Healthy	1591	Tomato	Spider mites	1676
			Tomato	Target spot	1404
			Tomato	Yellow leaf curl virus	3208
			Tomato	Mosaic virus	373

predicted diseased fraction of the leaf; the classifier sees it as an input, and the app reports it as severity (computed at 56×56).

We detach the gate on purpose. If gradients from the classification loss could reach it, the gate would be free to grow over any region that helps separate the classes, background included, and the map would no longer follow the disease.

E. Training objective and augmentation

The loss is

$$\mathcal{L} = \mathrm{CE}_{\kappa}(y, \hat{y}) + \lambda\, w\, \frac{1}{|\Omega|} \sum_{u \in \Omega} \beta_{m(u)}\, \mathrm{CE}\big(m(u), P(u)\big), \quad (3)$$

where CE_{κ} is cross-entropy with class weights $\kappa_c = \mathrm{clip}(N/(KN_c), 0.5, 3)$ to compensate for class imbalance, $\lambda = 1$, and $\beta_{\text{lesion}} = 3$ with $\beta = 1$ for the other labels, since lesion pixels are rare.

Image and mask are augmented together with random horizontal and vertical flips, 90° rotations, and jitter in brightness (±0.12), contrast and saturation ($\times[0.85, 1.15]$) and hue (±0.02). In addition, with probability 0.5 we replace every background pixel with a synthetic surface made of a random colour, a linear gradient and Gaussian noise. This background swap breaks the link between surface and class during training.

IV. EXPERIMENTAL SETUP

A. Data

In-domain. We use the 15 pepper, potato and tomato classes of PlantVillage [1] (Table I), resized to 224×224. All LesionNet variants and all evaluations share one stratified 70/15/15 split with a fixed seed.

Cross-dataset. PlantDoc [16] consists of field and internet photographs. Twelve of its classes match ours; it has no potato-healthy, target-spot or spider-mite class. We downloaded every image of the twelve classes from the public repository, from both its train and test folders, and 1,012 of them could be read. They are used only for testing, resized to 224×224 without cropping. Chance on this 12-way task is 8.3%.

B. Models

Baselines. MobileNetV2 [25], ResNet50 [27], EfficientNetB0 [24] and DenseNet121 [28] were each fine-tuned from ImageNet weights with a GAP–BN–dense head. Training ran for 5 epochs with the backbone frozen (lr 10^{-4}) and then 15 epochs with the last 50 layers unfrozen (lr 10^{-5}). These models were trained before this study on their own 70/15/15 split, so some of our test images may have been in their training data. If anything, this inflates their clean accuracy.

LesionNet variants. All variants were trained the same way on our split. The backbone was frozen for 3 epochs (Adam [29], lr 10^{-3}), after which the whole network was trained for 15 epochs with a cosine schedule [30] decaying from 10^{-4} to 10^{-6}. We used batch size 32, frozen BatchNorm statistics, mixed precision [31], early stopping with patience 5, and kept the checkpoint with the best validation accuracy.

- **A3, LesionNet**: EfficientNetB0 backbone with all components.
- **A0, Plain-EffNet**: the same network with global average pooling in place of the gate, λ=0 and no background swap. Backbone, split, schedule and training code are identical, so it serves as a like-for-like baseline.
- **A5, LesionNet-Mobile**: a MobileNetV2 backbone. A GPU driver fault ended its training after epoch 10 of 18, and we use the best checkpoint from that run (96.2% validation accuracy).

All training ran on a single NVIDIA RTX 4050 Laptop GPU (6 GB) under WSL2 with TensorFlow/Keras 3. A3 took 76 minutes and A0 took 42.

C. Background interventions

From the pseudo leaf mask of each test image we build five versions of it (Fig. 2):

TABLE II
ACCURACY (%) ON THE PLANTVILLAGE TEST SPLIT (3,109 IMAGES) UNDER BACKGROUND INTERVENTIONS. BG.-ONLY CHANCE: 6.7%. SHADED: OURS. BEST VALUES IN BOLD.

Model	Clean	F1	Grey	Swapped	Silh.↓	Bg. only↓
MobileNetV2	93.0	.919	76.0	71.6	25.5	19.1
ResNet50	**99.8**	**.998**	45.1	60.0	33.6	37.4
EfficientNetB0	96.4	.963	81.2	62.3	26.2	**17.8**
DenseNet121	96.2	.962	56.4	57.6	30.1	28.9
A0 Plain-EffNet	99.6	.997	67.5	63.7	47.2	32.7
A5 LesionNet-Mobile	96.0	.959	94.2	73.2	26.5	20.3
A3 LesionNet	99.5	.995	**98.7**	**81.8**	27.2	19.5

TABLE III
FAITHFULNESS OF THE EVIDENCE ON 2,222 DISEASED TEST IMAGES, AND SEVERITY. MAP: GRAD-CAM++ FOR CNNS, OWN LESION MAP FOR LESIONNET. A0 IS OMITTED (ITS SEGMENTATION HEAD IS UNTRAINED).

Model	Lesion↑	Bg.↓	Focus↑	Point.↑	IoU↑	Sev. ρ ↑
MobileNetV2	.094	.288	0.96	.492	.109	–
ResNet50	.087	.381	1.06	.636	.117	–
EfficientNetB0	.121	.247	1.22	.630	.133	–
DenseNet121	.071	.487	1.03	.604	.081	–
A5 LesionNet-Mobile	.340	.082	4.12	.949	.410	.784
A3 LesionNet	**.441**	**.067**	**6.06**	**.971**	**.505**	**.900**

- *clean*;
- *grey*: background set to 50% grey;
- *swapped*: the leaf pasted onto the full photo of a test image from a *different* class (part of that photo's own leaf can remain visible);
- *silhouette*: leaf set to grey, background kept;
- *background only*: the leaf filled in with resampled, blurred pixels from the image's own background, leaving neither leaf nor silhouette.

If a model relies on the leaf, it should keep its accuracy on the grey and swapped versions and drop to chance (6.7%) when only background is left.

D. Faithfulness and severity metrics

We explain each CNN with Grad-CAM++ for its predicted class, taken at the last convolutional block and upsampled bilinearly to 224×224. For LesionNet we use P_{lesion}. The evaluation covers the 2,222 diseased test images whose pseudo-mask is reliable and contains at least 1% lesion, and reports:

- the share of map mass on lesion and on background pixels;
- the *focus ratio*, i.e. mean map value on lesions divided by mean value on the whole leaf (1 = no preference);
- the *pointing game* [17]: the map's maximum lies within 15 px of a lesion pixel;
- the IoU between the map thresholded at 0.5 and the lesion mask.

For severity we report Spearman's ρ and the mean absolute error (MAE) between the predicted and pseudo-mask lesion percentages, along with the average predicted lesion percentage on the 487 healthy test images. Confidence intervals for accuracy use the Wilson score [32]; with $n = 3{,}109$ they are never wider than ± 1.8 points.

V. RESULTS

A. Accuracy under background interventions

Table II and Fig. 3 give the main result. Apart from MobileNetV2, every model is above 96% on clean images, yet ranking them by clean accuracy (ResNet50 first) gives almost the reverse of their ranking by robustness. ResNet50 loses 54.7 points when the background turns grey, and from background pixels alone it still names the right class in 37.4% of images, 5.6 times chance. All four CNNs stay well above chance on background-only images (17.8–37.4%), which confirms a strong background shortcut in this benchmark [7].

On clean images LesionNet is as accurate as the best CNN (99.5% against 99.8%, with overlapping confidence intervals). On grey backgrounds it loses 0.8 points and scores 98.7% (CI 98.3–99.1). That is an error rate of 1.3%, compared with 18.8% for EfficientNetB0, the most robust CNN. The swapped test is harder because another leaf can show up in the pasted background; there LesionNet reaches 81.8% and the CNNs 57.6–71.6%.

B. Ablation: where does the robustness come from?

A0 uses the same backbone, split, schedule and code as LesionNet and is just as accurate on clean images (99.6%). Yet it reaches only 67.5% on grey backgrounds, 31.2 points below LesionNet and 25 times its error, along with 63.7% on swapped backgrounds and 32.7% from background alone (LesionNet: 19.5%). A0 also gets 47.2% right from the silhouette, so it is using leaf shape, which gives away the crop. The robustness therefore comes from the LesionNet components and not from the backbone or the training recipe. The lighter model behaves the same way: LesionNet-Mobile keeps 94.2% on grey backgrounds, compared with 76.0% for the plain MobileNetV2 CNN, with similar clean accuracy (96.0% and 93.0%).

Three further ablations are implemented in the released code but were not run for this paper: background swap on its own, segmentation loss with global pooling, and the gate without the stop-gradient.

C. Does the evidence lie on the disease?

Table III and Fig. 4 suggest that the CNN explanations barely distinguish disease from healthy tissue. Focus ratios of 0.96–1.22 mean a lesion pixel gets about the same attention as a healthy one, and 25–49% of the map mass sits on the background. The peak of the map lands on a lesion in 49–64% of images. In our earlier study of 600 images, a random point on the leaf hit a lesion 53% of the time, so the peaks do little better than chance. LesionNet puts 44% of its evidence on lesions and 6.7% on background. Its peak falls on a lesion in 97% of images, and its map overlaps the lesion mask with an IoU of 0.50, against 0.08–0.13 for the CNNs.

Caveat. These metrics are computed against the same automatic masks LesionNet is trained on, which gives it an



Fig. 2. Background interventions on a test image (tomato early blight). (b) Automatic pseudo-mask: green = healthy leaf, red = lesion. (c) Leaf on 50% grey. (d) Leaf pasted onto a photo of another class (pepper, healthy). (e) Leaf removed, silhouette kept. (f) Leaf filled with the image's own background texture.

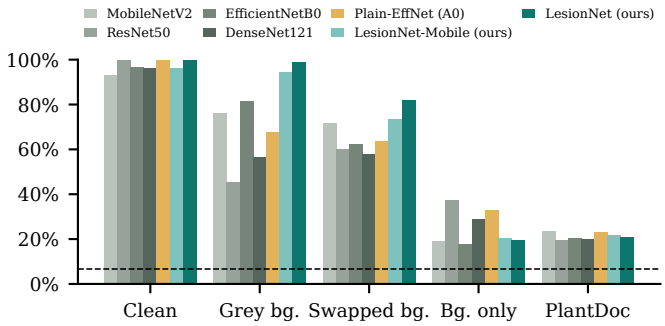


Fig. 3. Accuracy under interventions and on PlantDoc field photos. Dashed line: 15-way chance (6.7%); PlantDoc chance is 8.3%. For "Bg. only", lower is better.

advantage. The background tests in Table II and the cross-dataset test do not use lesion masks at test time (only the leaf mask is needed to build the variants), so they are the fairer comparisons.

D. Severity estimation

The lesion fraction predicted by LesionNet correlates with the measured fraction at $\rho = 0.900$, with an MAE of 5.7 percentage points, and on healthy leaves it predicts 0.55% diseased area on average. LesionNet-Mobile reaches $\rho = 0.784$ (MAE 10.3, 1.3% on healthy leaves). In our earlier 600-image study, by contrast, the "affected area" derived from Grad-CAM (the share of the leaf with attention above 0.25), a measure many applications use, had no relationship with lesion area ($|\rho| \leq 0.13$ for all four CNNs) and reported 66–100% affected area on completely healthy leaves.

E. Cross-dataset generalisation (negative result)

Field images are a different story (Table IV). Every model, LesionNet included, scores between 19.6% and 23.6% on the 12 classes, and the gaps between models are within the confidence interval of about ± 2.6 points. The models usually get the crop right (62–71% of images) but rarely the disease. Fig. 5 shows why. Field photos contain overlapping leaves, soil, hands, glare and blur. LesionNet's segmentation head, which only ever saw single leaves on plain surfaces, marks sunlit soil and blurred leaves as lesions: on healthy PlantDoc leaves it predicts 16.5% lesion, compared with 0.55% on PlantVillage. Removing the background shortcut is needed, then, but it is not enough. What remains is a shift in how leaves and lesions themselves look in the field.

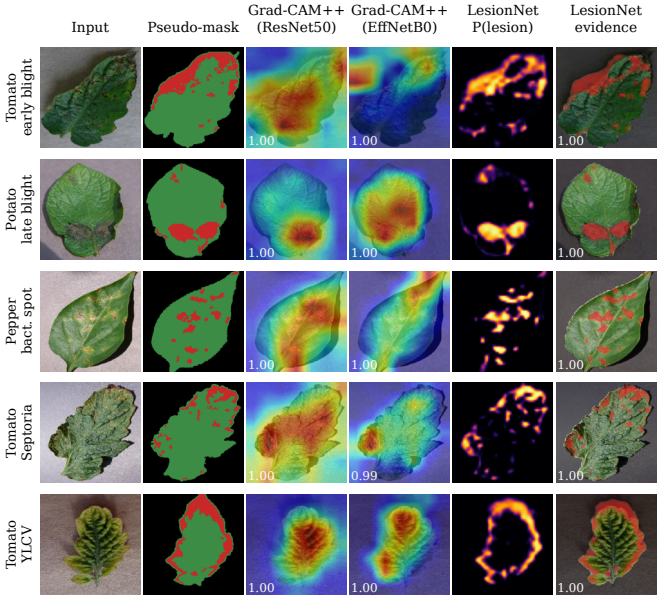


Fig. 4. Test images. Grad-CAM++ maps of the CNNs are coarse and often centred on healthy tissue or background. LesionNet's lesion probability and the evidence its decision is computed from follow individual spots. Numbers: predicted-class confidence (all predictions correct). Last row: the lesion map of yellow leaf curl virus follows the chlorotic margins.

TABLE IV
CROSS-DATASET TEST ON 1,012 PLANTDOC FIELD IMAGES (12 SHARED CLASSES, CHANCE 8.3%). NO MODEL SAW PLANTDOC DURING TRAINING. 95% CI $\approx \pm 2.6$ POINTS.

Model	Accuracy (%)	Macro-F1	Correct crop (%)
MobileNetV2	**23.6**	.189	68.9
ResNet50	19.6	.147	65.2
EfficientNetB0	20.2	.148	68.9
DenseNet121	20.1	.158	67.4
A0 Plain-EffNet	23.1	**.191**	70.4
A5 LesionNet-Mobile	21.6	.154	62.5
A3 LesionNet	20.8	.172	**70.7**

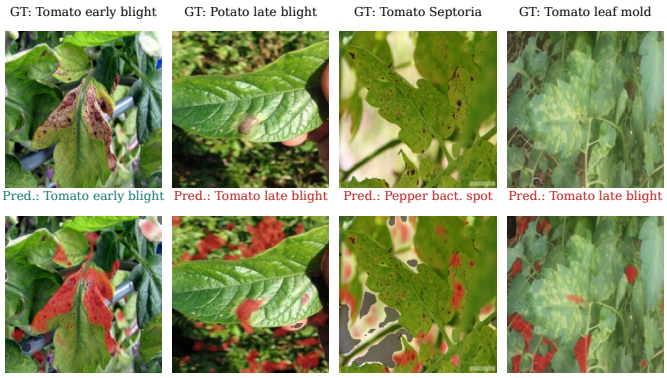


Fig. 5. LesionNet on PlantDoc field images (top: input with ground truth; bottom: lesion evidence and prediction). Single dominant leaves with clear lesions are handled (left); sunlit soil, overlapping leaves and blur are mistaken for lesions (right).

TABLE V
MODEL COST. PARAMETERS; BATCH-1 FLOAT32 LATENCY ON A 2-CORE X86 CPU (MEDIAN OF 20 RUNS, TENSORFLOW GRAPH MODE).

Model	Params (M)	CPU latency (ms)
MobileNetV2	2.63	26.2
ResNet50	24.78	96.8
EfficientNetB0	4.39	50.3
DenseNet121	7.34	75.9
A5 LesionNet-Mobile	3.35	33.1
A3 LesionNet	4.95	50.7

F. Computational cost

The decoder and gated head add 0.56 M parameters to EfficientNetB0 (13% more), and in our measurement no noticeable latency (50.7 against 50.3 ms, Table V). LesionNet-Mobile runs in 33 ms on a CPU, which makes on-device use realistic. Compared with ResNet50, LesionNet has a fifth of the parameters and about half the latency.

G. Implementation pitfall: Grad-CAM on nested Keras 3 models

In Keras 3, calling a backbone's `preprocess_input` on a symbolic tensor adds graph operations rather than layers. A common way of building the Grad-CAM gradient model replays `model.layers` up to the backbone, and so it silently drops that preprocessing. In our original code the gradient model agreed with the real model's class on only 22 of 60 images for ResNet50 and 4 of 60 for DenseNet121, and for DenseNet121 the heatmap correlation fell to 0.15. EfficientNetB0 and MobileNetV2 were not affected, since their preprocessing is either built in or absent. Every result in this paper uses a corrected builder that reproduces each model's softmax output exactly (maximum difference 0.0). Anyone comparing backbones with Grad-CAM should run the same check.

VI. DISCUSSION

Why the gate helps. A classifier with global pooling can weight any position in the image, and when the surface differs between classes the background is the easiest thing to learn. Lesion-gated pooling takes background positions out of the classifier input, and detaching the gate stops the classifier from widening it to get them back. The background swap attacks the same problem from the segmentation side by breaking the link between surface and class.

Why some background signal remains. LesionNet still classifies 19.5% of background-only images correctly, against 6.7% chance. We see two likely causes. Features at leaf positions have receptive fields that include nearby background, so some context leaks into F near the leaf edge. In addition, on background-only images the segmentation head may mistake some background texture for leaf. Reducing the receptive field near the boundary or erasing the background before the backbone are obvious things to try next, though both give up context in exchange for invariance.

Accuracy and understanding. In our study the most accurate model, ResNet50 at 99.8%, was also the least robust, with 45.1% on grey backgrounds. We would encourage PlantVillage benchmarks to report intervention accuracy next to clean accuracy. The tests are cheap, since they only need automatic leaf masks.

Limitations.

1) The pseudo-masks are colour-based and miss diffuse symptoms such as mosaic, leaf-curl virus and mite damage. The faithfulness numbers in Table III depend on these masks, so pixel-level claims would need hand-drawn masks such as PlantSeg [20].
2) Each configuration was trained once, with a single seed, and A5 stopped early at epoch 10 of 18. Our confidence intervals cover the sampling of the test set but not variation between training runs.
3) The CNN baselines were trained on a different split and may have seen some of our test images, which would push their clean accuracy up. The A0 baseline does not have this problem.
4) No model we tested generalises to field images (Table IV).

Future work. The most pressing next step is fine-tuning on the PlantDoc training split, with pseudo-masks from the same pipeline, to see how quickly each model adapts. Beyond that, we plan to work on segmentation of several leaves per image, run the remaining ablations with three seeds each, and evaluate against hand-annotated lesion masks.

VII. DEPLOYMENT IN AGRI-X

LesionNet is the default diagnostic model in AGRI-X, an open-source Streamlit app we built for farmers in India (Fig. 6). A single forward pass returns the class, the lesion map, the diseased share of the leaf and statistics for each spot. The disease and the severity grade decide which treatment advice is shown, and a spread-risk score is computed from live weather forecasts. The app also recommends crops for the user's location, estimates yield and includes a simple farmer assistant, and one page reproduces the robustness results from this paper. Since the map on screen is the classifier's actual

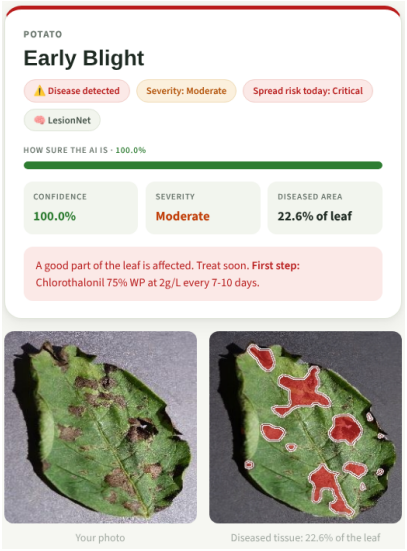


Fig. 6. LesionNet in the AGRI-X application: the diagnosis, the confidence, a severity grade from the predicted diseased area (mild $< 10\%$, moderate 10–25%, severe $> 25\%$) and the lesion map used for the decision.

evidence, a farmer who sees red on the table instead of the leaf has a concrete reason not to trust that diagnosis.

VIII. Conclusion

Near-perfect accuracy on PlantVillage does not mean a model is looking at the disease. The four CNNs we studied put most of their evidence on healthy tissue and background, and the most accurate one lost more than half its accuracy when the background was changed. LesionNet restricts the classifier to what its own segmentation head marks as leaf or lesion. It needs no manual annotation, matches the best CNN on clean images, hardly changes when the background is replaced, and its lesion map is both a faithful explanation and a severity measure. Field photos remain unsolved for every model we tried. We release our code, test protocol and negative results so that progress on that problem can be measured.

Data and Code Availability

Code, trained LesionNet weights, evaluation scripts and all result files are available at https://github.com/raj06872436/AI_Farming_Project (directory `research/lesion_net`). PlantVillage and PlantDoc are public datasets.

Acknowledgment

The authors thank the Department of Information Technology, Dwarkadas J. Sanghvi College of Engineering, Mumbai, for its support.

References

[1] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," *arXiv preprint arXiv:1511.08060*, 2015.

[2] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.

[3] K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018.

[4] E. C. Too, L. Yujian, S. Njuki, and L. Yingchun, "A comparative study of fine-tuning deep learning models for plant disease identification," *Computers and Electronics in Agriculture*, vol. 161, pp. 272–279, 2019.

[5] J. G. A. Barbedo, "Factors influencing the use of deep learning for plant disease recognition," *Biosystems Engineering*, vol. 172, pp. 84–91, 2018.

[6] ——, "Impact of dataset size and variety on the effectiveness of deep learning and transfer learning for plant disease classification," *Computers and Electronics in Agriculture*, vol. 153, pp. 46–53, 2018.

[7] M. A. Noyan, "Uncovering bias in the PlantVillage dataset," *arXiv preprint arXiv:2206.04374*, 2022.

[8] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665–673, 2020.

[9] K. Xiao, L. Engstrom, A. Ilyas, and A. Madry, "Noise or signal: The role of image backgrounds in object recognition," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.

[10] A. S. Ross, M. C. Hughes, and F. Doshi-Velez, "Right for the right reasons: Training differentiable models by constraining their explanations," in *Proc. 26th Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2017, pp. 2662–2670.

[11] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 618–626.

[12] A. Chattopadhay, A. Sarkar, P. Howlader, and V. N. Balasubramanian, "Grad-CAM++: Generalized gradient-based visual explanations for deep convolutional networks," in *Proc. IEEE Winter Conf. Applications of Computer Vision (WACV)*, 2018, pp. 839–847.

[13] J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, and B. Kim, "Sanity checks for saliency maps," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[14] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, 2019.

[15] G. Wang, Y. Sun, and J. Wang, "Automatic image-based plant disease severity estimation using deep learning," *Computational Intelligence and Neuroscience*, vol. 2017, p. 2917536, 2017.

[16] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, and N. Batra, "PlantDoc: A dataset for visual plant disease detection," in *Proc. 7th ACM IKDD CoDS and 25th COMAD*, 2020, pp. 249–253.

[17] J. Zhang, Z. Lin, J. Brandt, X. Shen, and S. Sclaroff, "Top-down neural attention by excitation backprop," in *Proc. European Conf. Computer Vision (ECCV)*, 2016, pp. 543–559.

[18] C. Chen, O. Li, D. Tao, A. Barnett, C. Rudin, and J. K. Su, "This looks like that: Deep learning for interpretable image recognition," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[19] H. Fukui, T. Hirakawa, T. Yamashita, and H. Fujiyoshi, "Attention branch network: Learning of attention mechanism for visual explanation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 10 705–10 714.

[20] T. Wei *et al.*, "PlantSeg: A large-scale in-the-wild dataset for plant disease segmentation," *arXiv preprint arXiv:2409.04038*, 2024.

[21] D.-H. Lee, "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks," *ICML 2013 Workshop on Challenges in Representation Learning*, 2013.

[22] C. Rother, V. Kolmogorov, and A. Blake, ""GrabCut": Interactive foreground extraction using iterated graph cuts," *ACM Transactions on Graphics*, vol. 23, no. 3, pp. 309–314, 2004.

[23] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.

[24] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. 36th Int. Conf. Machine Learning (ICML)*, 2019, pp. 6105–6114.

[25] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[26] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2117–2125.

[27] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
[28] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 4700–4708.
[29] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.
[30] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2017.
[31] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2018.
[32] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.